

3. Graphs and Trees

Contents

Bellman Ford	1
Floyd Tricks	2
Topological Sort	3
Bipartite Check	3
Dijkstra	4
Tree Diameter	4
Tree	5
Tree Hash (rooted/unrooted)	6
Binary Lifting	7
LCA	7
DSU on Trees	8
SCC / Strongly Connected Componenets	9
WLCA	11
Rerooter	12
Dynamic Connectivity	13
Max Flow (Dinik)	15
Max Bipartite Matching (Karp) [with building]	16

Bellman Ford

```
1 // Computes Single-Source Shortest Paths (SSSP) on graphs that
2 // contain NEGATIVE edge weights.
3 // It explicitly detects negative weight cycles reachable from the source vertex.
4 // - Average Time Complexity:  $O(E)$  where  $E$  is the number of edges.
5 // - Worst Case Time Complexity:  $O(V * E)$ .
6 // vector<int> dist;
7 // bool ok = spfa(0, adj, dist);
8
9 const int INF = 1e9;
10 bool spfa(int s, const vector<vector<pair<int, int>>>& adj, vector<int>& d) {
11     int n = adj.size();
```

```

12     d.assign(n, INF);
13     vector<int> cnt(n, 0);
14     vector<int> inqueue(n, 0);
15     queue<int> q;
16
17     d[s] = 0;
18     q.push(s);
19     inqueue[s] = 1;
20
21     while (!q.empty()) {
22         int v = q.front();
23         q.pop();
24         inqueue[v] = 0;
25         for (const auto& [to, len] : adj[v]) {
26             if (d[v] + len < d[to]) {
27                 d[to] = d[v] + len;
28                 if (!inqueue[to]) {
29                     q.push(to);
30                     inqueue[to] = 1;
31                     if (++cnt[to] > n) return false; // negative cycle
32                 }
33             }
34         }
35     }
36     return true;
37 }

```

Floyd Tricks

```

1 void TransitiveClosure(int n, vector<vector<int>>& adj) {
2     // 0 = disconnected, 1 = connected
3     for (int k = 0; k < n; ++k)
4         for (int i = 0; i < n; ++i)
5             for (int j = 0; j < n; ++j)
6                 adj[i][j] |= (adj[i][k] & adj[k][j]);
7 }
8
9 void minimax(int n, vector<vector<int>>& adj) {
10    // Path such that max value on road is minimized
11    for (int k = 0; k < n; ++k)
12        for (int i = 0; i < n; ++i)
13            for (int j = 0; j < n; ++j)
14                adj[i][j] = min(adj[i][j], max(adj[i][k], adj[k][j]));
15 }
16
17 void maximin(int n, vector<vector<int>>& adj) {
18    // Path such that min value on road is maximized
19    for (int k = 0; k < n; ++k)
20        for (int i = 0; i < n; ++i)
21            for (int j = 0; j < n; ++j)
22                adj[i][j] = max(adj[i][j], min(adj[i][k], adj[k][j]));
23 }
24
25 void longestPathDAG(int n, vector<vector<int>>& adj) {
26    // Only works for DAGs (no positive cycles)

```

```

27     for (int k = 0; k < n; ++k)
28         for (int i = 0; i < n; ++i)
29             for (int j = 0; j < n; ++j)
30                 adj[i][j] = max(adj[i][j], max(adj[i][k], adj[k][j]));
31 }
32
33 void countPaths(int n, vector<vector<int>>& adj) {
34     // Floyd-Warshall for counting number of paths
35     for (int k = 0; k < n; ++k)
36         for (int i = 0; i < n; ++i)
37             for (int j = 0; j < n; ++j)
38                 adj[i][j] += adj[i][k] * adj[k][j];
39 }
40
41 bool isNegativeCycle(int n, const vector<vector<int>>& adj) {
42     // run floyd first
43     for (int i = 0; i < n; ++i)
44         if (adj[i][i] < 0)
45             return true;
46     return false;
47 }
48
49 bool anyEffectiveCycle(int n, const vector<vector<int>>& adj,
50     int src, int dest, int 00) {
51     // run floyd first
52     for (int i = 0; i < n; ++i)
53         if (adj[i][i] < 0 && adj[src][i] < 00 && adj[i][dest] < 00)
54             return true;
55     return false;
56 }

```

Topological Sort

```

1  queue<int> queue;
2  for (int i = 0; i < n; i++)
3      if (in_degree[i] == 0) { queue.push(i); }
4
5  vector<int> top_sort;
6  while (!queue.empty()) {
7      int curr = queue.front();
8      queue.pop();
9      top_sort.push_back(curr);
10     for (int next : graph[curr]) {
11         if (--in_degree[next] == 0) { queue.push(next); }
12     }
13 }

```

Bipartite Check

```

1  int n;
2  vector<vector<int>> adj;
3
4  vector<int> side(n, -1);
5  bool is_bipartite = true;

```

```

6  queue<int> q;
7  for (int st = 0; st < n; ++st) {
8      if (side[st] == -1) {
9          q.push(st);
10         side[st] = 0;
11         while (!q.empty()) {
12             int v = q.front();
13             q.pop();
14             for (int u : adj[v]) {
15                 if (side[u] == -1) {
16                     side[u] = side[v] ^ 1;
17                     q.push(u);
18                 } else {
19                     is_bipartite &= side[u] != side[v];
20                 }
21             }
22         }
23     }
24 }

```

Dijkstra

```

1  vector<ll> dijkstra(int src, vector<int> &parent) {
2      vector<ll> dist(n + 1, INF);
3      parent.assign(n + 1, -1);
4      priority_queue<pair<ll, int>, vector<pair<ll, int> >, greater<pair<ll, int> > >
5          pq;
6      dist[src] = 0;
7      pq.push({0, src});
8      while (!pq.empty()) {
9          auto [d, v] = pq.top();
10         pq.pop();
11         if (d != dist[v]) continue;
12         for (auto [u, w]: graph[v]) {
13             if (dist[u] > d + w) {
14                 dist[u] = d + w;
15                 parent[u] = v;
16                 pq.push({dist[u], u});
17             }
18         }
19     }
20     return dist;
21 }

```

Tree Diameter

```

1  pair<int, int> dfs(int u, int p) {
2      pair<int, int> ret = {0, u};
3      for (int v : tree[u]) {
4          if (v == p) continue;
5          auto x = dfs(v, u);
6          ret = max(ret, {x.first + 1, x.second});
7      }
8      return ret;

```

```
9 }
```

Tree

```
1 struct tree {
2     int root = 0;
3     vector<vector<int>> g;
4     explicit tree(int n) : g(n) { }
5     void add(int u, int v) {
6         g[u].push_back(v);
7         g[v].push_back(u);
8     }
9
10    vector<int> &operator[](int u) {
11        return g[u];
12    }
13
14    int cntDfs = 0;
15    vector<int> in, out, lvl, sz, top, par, seq;
16
17    void init(int rt = 0) {
18        root = rt;
19        in = out = lvl = top = par = seq = vector<int>(g.size());
20        sz.resize(g.size(), 1);
21        par[root] = top[root] = root;
22        dfs(root);
23        dfs2(root);
24    }
25
26    void dfs(int u) {
27        for(int &v : g[u]) {
28            lvl[v] = lvl[u] + 1;
29            par[v] = u;
30            g[v].erase(find(g[v].begin(), g[v].end(), u));
31            dfs(v);
32            sz[u] += sz[v];
33            if(sz[v] > sz[g[u][0]])
34                swap(v, g[u][0]);
35        }
36    }
37    void dfs2(int u) {
38        in[u] = cntDfs++;
39        seq[in[u]] = u;
40        for(int v : g[u]) {
41            top[v] = v == g[u][0]? top[u]: v;
42            dfs2(v);
43        }
44        out[u] = cntDfs - 1;
45    }
46
47    int jump(int u, int k) {
48        if(k > lvl[u]) return -1;
49
50        int d = lvl[u] - k;
51        while (lvl[top[u]] > d) {
52            u = par[top[u]];
```

```

53     }
54
55     return seq[in[u] - lvl[u] + d];
56 }
57
58 bool isAncestor(int u, int v) {
59     return in[u] <= in[v] && in[v] <= out[u];
60 }
61
62 int lca(int u, int v) {
63     if(lvl[u] > lvl[v]) swap(u, v);
64     if(isAncestor(u, v)) return u;
65     while (top[u] != top[v]) {
66         if (lvl[top[u]] > lvl[top[v]]) {
67             u = par[top[u]];
68         } else {
69             v = par[top[v]];
70         }
71     }
72     return lvl[u] < lvl[v]? u: v;
73 }
74
75 int dis(int u, int v) {
76     return lvl[u] + lvl[v] - 2 * lvl[lca(u, v)];
77 }
78 };

```

Tree Hash (rooted/unrooted)

```

1  // tree hash
2  using u64 = uint64_t;
3  mt19937_64 rng = []{
4      u64 time_entropy = chrono::steady_clock::now().time_since_epoch().count();
5      u64 memory_entropy = (uintptr_t)make_unique<char>().get();
6      seed_seq ss{time_entropy, memory_entropy};
7      return mt19937_64(ss);
8  }();
9  u64 SEED = rng();
10 u64 mix(u64 x) {
11     x += SEED + 0x9e3779b97f4a7c15;
12     x = (x ^ x >> 30) * 0xbf58476d1ce4e5b9;
13     x = (x ^ x >> 27) * 0x94d049bb133111eb;
14     return x ^ x >> 31;
15 }
16 u64 treehash(int u, int p, vector<vector<int>>& t) {
17     u64 ret = 1;
18     // do ret = ret * P + mix() if the order is important
19     for (int v : t[u]) if (v ^ p)
20         ret += mix(treehash(v, u, t));
21     return ret;
22 }
23 // unrooted: find centroids and treat them as roots
24 u64 utreehash(vector<vector<int>>& t) {
25     int n = t.size();
26     vector<int> sz(n), cents;

```

```

27     function<void(int, int)> dfs1 = [&](int u, int p) {
28         sz[u] = 1;
29         bool can = 1;
30         for (int v : t[u]) if (v ^ p) {
31             dfs1(v, u);
32             if (sz[v] * 2 > n) can = 0;
33             sz[u] += sz[v];
34         }
35         if (n - sz[u] > n/2) can = 0;
36         if (can) cents.push_back(u);
37     };
38     dfs1(0, 0);
39     return min(treeshash(cents.front(), -1, t), treeshash(cents.back(), -1, t));
40 }

```

Binary Lifting

```

1  int Log = __lg(n) + 1;
2  vector<vector<int>> lift(n+1, vector<int>(Log, -1));
3  function<void(int, int)> dfs2 = [&](int u, int p) {
4      for (auto v : tree[u]) if (v != p) {
5          lift[v][0] = u;
6          for (int l = 1; l < Log && ~lift[v][l-1]; l++)
7              lift[v][l] = lift[ lift[v][l-1] ][l-1];
8          dfs2(v, u);
9      }
10 };
11 dfs2(root, -1);
12
13 if (depth[u] < depth[v])
14     swap(u, v);
15 int k = depth[u] - depth[v];
16
17 for (int l = Log-1; ~l && ~u; l--) {
18     if (k >> l & 1)
19         u = lift[u][l];
20 }
21 if (u == v) return u;
22 for (int l = Log-1; l > -1; l--) {
23     if (lift[u][l] != lift[v][l])
24         u = lift[u][l], v = lift[v][l];
25 }
26 return lift[u][0];

```

LCA

```

1  struct LCA {
2      int n, m, LOG;
3      vector<vector<int>> g;
4      vector<int> euler, first, depth, lg;
5      vector<vector<int>> st;
6
7      LCA(const vector<vector<int>>& tree, int root) {
8          n = tree.size();

```

```

9      g = tree;
10     first.assign(n, -1);
11     depth.assign(n, 0);
12
13     dfs(root, -1, 0);
14     m = euler.size();
15
16     lg.assign(m+1, 0);
17     for (int i = 2; i <= m; i++)
18         lg[i] = lg[i>>1] + 1;
19     LOG = lg[m] + 1;
20
21     st.assign(m, vector<int>(LOG));
22     for (int i = 0; i < m; i++)
23         st[i][0] = euler[i];
24
25     for (int j = 1; j < LOG; j++) {
26         for (int i = 0; i + (1<<j) <= m; i++) {
27             int x = st[i][j-1], y = st[i + (1<<(j-1))][j-1];
28             st[i][j] = (depth[x] < depth[y] ? x : y);
29         }
30     }
31 }
32
33 void dfs(int u, int p, int d) {
34     if (first[u] == -1) first[u] = euler.size();
35     euler.push_back(u);
36     depth[u] = d;
37     for (int v : g[u]) {
38         if (v == p) continue;
39         dfs(v, u, d+1);
40         euler.push_back(u);
41     }
42 }
43
44 int lca(int u, int v) {
45     int L = first[u], R = first[v];
46     if (L > R) swap(L, R);
47     int len = R - L + 1, k = lg[len];
48     int x = st[L][k], y = st[R - (1<<k) + 1][k];
49     return depth[x] < depth[y] ? x : y;
50 }
51
52 int dis(int u, int v) {
53     return depth[u] + depth[v] - 2 * depth[lca(u, v)];
54 }
55 };

```

DSU on Trees

```

1 void pre(int u) {
2     sz[u] = 1;
3     for (int &v : tree[u]) {
4         tree[v].erase(find(tree[v].begin(), tree[v].end(), u));
5         pre(v);

```

```

6         sz[u] += sz[v];
7         if (sz[v] > sz[tree[u].front()]) swap(v, tree[u].front());
8     }
9 }
10
11 void addver(int u) {
12
13 }
14 void removever(int u) {
15
16 }
17
18 void addsub(int u) {
19     addver(u);
20     for (int v : tree[u]) addsub(v);
21 }
22 void removesub(int u) {
23     removever(u);
24     for (int v : tree[u]) removesub(v);
25 }
26
27 void dfs(int u) {
28     for (int v : tree[u]) if (v != tree[u].front()) {
29         dfs(v);
30         removesub(v);
31     }
32     if (!tree[u].empty())
33         dfs(tree[u].front());
34     addver(u);
35     for (int v : tree[u])
36         if (v != tree[u].front())
37             addsub(v);
38
39     // query[u]
40 }

```

SCC / Strongly Connected Componenets

```

1 // --- USAGE REQUIREMENTS ---
2 // 1. timer must be declared and initialized to 0 outside the function.
3 // 2. g is your 0-based directed adjacency list (vector<vector<int>>).
4 // 3. v is a 0-based array/vector of vertex weights
5     (used to find the min weight per SCC).
6 // 4. idsccl[i] will hold the 0-based SCC ID for vertex i.
7 // 5. cond generates the condensed DAG where each node is a distinct SCC.
8
9 // --- MODIFICATIONS FOR BRIDGES & ARTICULATION POINTS (UNDIRECTED GRAPH) ---
10 // 1. Update the lambda signature to pass the parent: tarj = [&](int u, int p = -1)
11 // 2. Add int children = 0; at the top of the lambda to
12     count root children for APs.
13 // 3. Ignore the back-edge to the parent in the loop: if (v_ == p) continue;
14 // 4. You can completely remove vis, stk, mn, and idsccl
15     arrays if you are not extracting SCCs.
16
17 vector<int> tin(n, -1), low(n), vis(n), stk, mn, idsccl(n, -1);

```

```

18 function<void(int)> tarj = [&](int u) { // Change to: (int u, int p = -1)
19     tin[u] = low[u] = timer++;
20     vis[u] = 1; // Marks u as currently in the active SCC stack
21     stk.push_back(u);
22
23     // int children = 0; // Uncomment for Articulation Points
24
25     for (int v_ : g[u]) {
26         // Uncomment to prevent traversing back to parent in undirected graphs
27         // if (v_ == p) continue;
28
29         if (tin[v_] == -1) {
30             // children++; // Uncomment for Articulation Points
31             tarj(v_); // Change to: tarj(v_, u)
32             low[u] = min(low[u], low[v_]);
33
34             // --- BRIDGE CHECK ---
35             // if (low[v_] > tin[u]) { /* Edge (u, v_) is a bridge */ }
36
37             // --- ARTICULATION POINT CHECK (NON-ROOT) ---
38             // if (low[v_] >= tin[u] && p != -1) { /* Vertex u is an AP */ }
39
40             } else if (vis[v_]) {
41                 // Back-edge found. v_ is already visited AND
42                 // still in the current stack
43                 low[u] = min(low[u], tin[v_]);
44             }
45     }
46
47     // --- SCC EXTRACTION LOGIC ---
48     // If u is the root of an SCC, pop all vertices belonging to
49     // it from the stack
50     if (low[u] == tin[u]) {
51         int t;
52         mn.push_back(1e5); // Initialize the minimum weight for this new SCC
53         do {
54             t = stk.back();
55             stk.pop_back();
56             vis[t] = 0; // Mark as removed from the active stack
57             // Track min value (requires your v array)
58             mn.back() = min(mn.back(), v[t]);
59             // Assign the new SCC ID to vertex t
60             idsc[t] = (int)mn.size() - 1;
61         } while (t ^ u);
62     }
63
64     // --- ARTICULATION POINT CHECK (ROOT) ---
65     // if (p == -1 && children > 1) { /* Vertex u (the root) is an AP */ }
66 };
67
68 for (int i = 0; i < n; i++)
69     if (tin[i] == -1)
70         tarj(i); // Change to: tarj(i, -1)
71
72 // --- BUILD THE CONDENSED DAG ---
73 vector<vector<int>> cond(mn.size());
74 for (int i = 0; i < n; i++)

```

```

75     for (int u : g[i])
76         if (idscc[i] ^ idscc[u]) // If an edge connects two different SCCs
77             cond[idscc[i]].push_back(idscc[u]);

```

WLCA

```

1  template<class T>
2  struct WLCA {
3      int n, Log;
4      vector<vector<int>>> up;
5      vector<vector<T>>> val;
6      vector<int> lvl;
7      explicit WLCA(vector<vector<pair<int, T>>> &g, int root = 0) : n((int)g.size()),
8          lvl(n), Log(__lg(n) + 1), up(n, vector<int>(Log, root)),
9          val(n, vector<T>(Log)) {
10         function<void(int)> dfs = [&](int u) -> void {
11             for(auto [v, w] : g[u]) {
12                 if(v == up[u][0]) continue;
13                 lvl[v] = lvl[u] + 1, up[v][0] = u, val[v][0] = w;
14                 for(int l = 1; l < Log; l++) {
15                     up[v][l] = up[up[v][l - 1]][l - 1];
16                     val[v][l] = val[v][l - 1] + val[up[v][l - 1]][l - 1];
17                 }
18                 dfs(v);
19             }
20         };
21         dfs(root);
22     }
23     pair<int, T> k_ancestor(int u, int k) {
24         T ans;
25         while(k) {
26             ans = ans + val[u][__builtin_ctz(k)];
27             u = up[u][__builtin_ctz(k)];
28             k &= k - 1;
29         }
30         return {u, ans};
31     }
32     int lca(int u, int v) {
33         if(lvl[u] < lvl[v]) swap(u, v);
34         u = k_ancestor(u, lvl[u] - lvl[v]).first;
35         if(u == v) return u;
36         for(int l = Log - 1; l >= 0; l--) {
37             if(up[u][l] ^ up[v][l]) {
38                 u = up[u][l], v = up[v][l];
39             }
40         }
41         return up[u][0];
42     }
43     int dis(int u, int v, int l = -1) {
44         if(l == -1) l = lca(u, v);
45         return lvl[u] + lvl[v] - 2 * lvl[l];
46     }
47 };

```

Rerooter

```
1 namespace reroot {
2     const auto exclusive = [] (const auto& a, const auto& base,
3                               const auto& merge_into, int vertex) {
4         int n = (int)a.size();
5         using Aggregate = decay_t<decltype(base)>;
6         vector<Aggregate> b(n, base);
7         for (int bit = __lg(n); bit >= 0; --bit) {
8             for (int i = n - 1; i >= 0; --i) b[i] = b[i >> 1];
9             int sz = n - (n & !bit);
10            for (int i = 0; i < sz; ++i) {
11                int index = (i >> bit) ^ 1;
12                b[index] = merge_into(b[index], a[i], vertex, i);
13            }
14        }
15        return b;
16    };
17    // MergeInto : Aggregate * Value * Vertex(int) * EdgeIndex(int) -> Aggregate
18    // Base : Vertex(int) -> Aggregate
19    // FinalizeMerge : Aggregate * Vertex(int) * EdgeIndex(int) -> Value
20    const auto rerooter = [] (const auto& g, const auto& base,
21                              const auto& merge_into, const auto& finalize_merge) {
22        int n = (int)g.size();
23        using Aggregate = decay_t<decltype(base(0))>;
24        using Value = decay_t<decltype(finish_merge(base(0), 0, 0))>;
25        vector<Value> root_dp(n), dp(n);
26        vector<vector<Value>> edge_dp(n), redge_dp(n);
27
28        vector<int> bfs, parent(n);
29        bfs.reserve(n);
30        bfs.push_back(0);
31        for (int i = 0; i < n; ++i) {
32            int u = bfs[i];
33            for (auto v : g[u]) {
34                if (parent[u] == v) continue;
35                parent[v] = u;
36                bfs.push_back(v);
37            }
38        }
39
40        for (int i = n - 1; i >= 0; --i) {
41            int u = bfs[i];
42            int p_edge_index = -1;
43            Aggregate aggregate = base(u);
44            for (int edge_index = 0; edge_index < (int)g[u].size(); ++edge_index) {
45                int v = g[u][edge_index];
46                if (parent[u] == v) {
47                    p_edge_index = edge_index;
48                    continue;
49                }
50                aggregate = merge_into(aggregate, dp[v], u, edge_index);
51            }
52            dp[u] = finalize_merge(aggregate, u, p_edge_index);
53        }
54    }
```

```

55     for (auto u : bfs) {
56         dp[parent[u]] = dp[u];
57         edge_dp[u].reserve(g[u].size());
58         for (auto v : g[u]) edge_dp[u].push_back(dp[v]);
59         auto dp_exclusive = exclusive(edge_dp[u], base(u), merge_into, u);
60         redge_dp[u].reserve(g[u].size());
61         for (int i = 0; i < (int)dp_exclusive.size(); ++i)
62             redge_dp[u].push_back(finalize_merge(dp_exclusive[i], u, i));
63         root_dp[u] = finalize_merge(n > 1 ?
64             merge_into(dp_exclusive[0], edge_dp[u][0], u, 0) :
65             base(u), u, -1);
66         for (int i = 0; i < (int)g[u].size(); ++i) {
67             dp[g[u][i]] = redge_dp[u][i];
68         }
69     }
70
71     return make_tuple(move(root_dp), move(edge_dp), move(redge_dp));
72 };
73 } // namespace reroot
74 // [&](Aggregate agg, Aggregate chdp, int v, int eid)
75 // [&](Aggregate agg, int v, int eid);

```

Dynamic Connectivity

```

1  struct Query {
2      char t;
3      int u, v;
4  };
5
6  struct Elem {
7      int u, v, szU, cnt;
8  };
9
10 struct DSURollback {
11     // offline : [+ a b] add edge between a, b
12     //           [- a b] remove edge between a, b
13     //           [?] number of connected components
14     int cnt;
15     stack<Elem> st;
16     vector<bool> ans;
17     vector<int> sz, par;
18     map<int, vector<pair<int, int>>> g;
19
20     DSURollback(int n) {
21         cnt = n;
22         par.resize(n + 1);
23         sz.resize(n + 1, 1);
24         iota(par.begin(), par.end(), 0);
25     }
26
27     void rollback(int x) {
28         while (st.size() > x) {
29             auto e = st.top();
30             st.pop();
31             cnt = e.cnt;
32             sz[e.u] = e.szU;

```

```

33         par[e.v] = e.v;
34     }
35 }
36
37 int findSet(int u) {
38     return par[u] == u ? u : findSet(par[u]);
39 }
40
41 void update(int u, int v) {
42     st.push({u, v, sz[u], cnt});
43     cnt--;
44     par[v] = u;
45     sz[u] += sz[v];
46 }
47
48 void unionSet(int u, int v) {
49     u = findSet(u);
50     v = findSet(v);
51     if (u != v) {
52         if (sz[u] < sz[v])
53             swap(u, v);
54         update(u, v);
55     }
56 }
57
58 void solve(int x, int l, int r) {
59     int cur = st.size();
60
61     for (auto i: g[x])
62         unionSet(i.first, i.second);
63
64     if (l == r) {
65         if (ans[l])
66             cout << cnt << endl;
67         rollback(cur);
68         return;
69     }
70     int m = (l + r) >> 1;
71     solve(x * 2, l, m);
72     solve(x * 2 + 1, m + 1, r);
73     rollback(cur);
74 }
75
76 void traverse(int x, int lX, int rX, int l, int r, int u, int v) {
77     if (rX < l || lX > r)
78         return;
79     if (lX >= l && rX <= r) {
80         g[x].emplace_back(u, v);
81         return;
82     }
83     int m = (lX + rX) >> 1;
84     traverse(x * 2, lX, m, l, r, u, v);
85     traverse(x * 2 + 1, m + 1, rX, l, r, u, v);
86 }
87
88 void build(vector<Query> &queries) {

```

```

89     int q = queries.size();
90     ans.resize(q);
91     map<pair<int, int>, vector<pair<int, int>>> mp;
92     for (int i = 0; i < queries.size(); i++) {
93         auto cur = queries[i];
94         if (cur.u > cur.v)
95             swap(cur.u, cur.v);
96         if (cur.t == '?')
97             ans[i] = 1;
98         else if (cur.t == '+')
99             mp[{cur.u, cur.v}].emplace_back(i, queries.size());
100        else {
101            mp[{cur.u, cur.v}].back().second = i - 1;
102            traverse(1, 0, q - 1, mp[{cur.u, cur.v}].back().first,
103                mp[{cur.u, cur.v}].back().second, cur.u, cur.v);
104        }
105    }
106
107    for (auto i: mp) {
108        for (auto j: i.second) {
109            if (j.second == q)
110                traverse(1, 0, q - 1, j.first, q - 1, i.first.first,
111                    i.first.second);
112        }
113    }
114 }
115 };
116
117 void testCase() {
118     int n, q;
119     cin >> n >> q;
120     if (!q)
121         return;
122     DSURollback dsu(n);
123     vector<Query> queries(q);
124     char t;
125     int x, y;
126     for (int i = 0; i < q; i++) {
127         cin >> t;
128         if (t == '?')
129             queries[i] = {t, 0, 0};
130         else {
131             cin >> x >> y;
132             if (y < x)
133                 swap(x, y);
134             queries[i] = {t, x, y};
135         }
136     }
137
138     dsu.build(queries);
139     dsu.solve(1, 0, q - 1);
140 }

```

Max Flow (Dinik)

```

1  class Dinic { //  $O(n^2 * m)$ , 0-based
2      private:
3          struct E {
4              int to, rev;
5              ll c, oc;
6              ll f() { return max(oc - c, 0LL); }
7          };
8          vector<int> lvl, ptr, q;
9          vector<vector<E>> > g;
10         ll dfs(int v, int t, ll f) {
11             if (v == t || !f) return f;
12             for (int &i = ptr[v]; i < g[v].size(); ++i) {
13                 E &e = g[v][i];
14                 if (lvl[e.to] == lvl[v] + 1 && e.c)
15                     if (ll p = dfs(e.to, t, min(f, e.c))) {
16                         e.c -= p;
17                         g[e.to][e.rev].c += p;
18                         return p;
19                     }
20             }
21             return 0;
22         }
23     public:
24         Dinic(int n) : lvl(n), ptr(n), q(n), g(n) {}
25         void add_edge(int u, int v, ll c) { // directed
26             g[u].push_back({v, (int)g[v].size(), c, c});
27             g[v].push_back({u, (int)g[u].size() - 1, 0, 0});
28         }
29         ll calc(int s, int t) { // source to destination
30             ll flow = 0;
31             while (true) {
32                 fill(lvl.begin(), lvl.end(), 0);
33                 int qs = 0, qe = 0;
34                 lvl[q[qe++] = s] = 1;
35                 while (qs < qe && !lvl[t]) {
36                     int v = q[qs++];
37                     for (auto &e : g[v])
38                         if (!lvl[e.to] && e.c) lvl[q[qe++] = e.to] = lvl[v] + 1;
39                 }
40                 if (!lvl[t]) break;
41                 fill(ptr.begin(), ptr.end(), 0);
42                 while (ll p = dfs(s, t, LLONG_MAX)) flow += p;
43             }
44             return flow;
45         }
46         void reset() { // before calling calc again
47             for (auto &adj : g)
48                 for (auto &e : adj) e.c = e.oc;
49         }
50         bool leftOfMinCut(int a) { return lvl[a] != 0; }
51     };
52 }

```

Max Bipartite Matching (Karp) [with building]

```

1 struct matching {
2     int nL, nr;
3     vector<vector<int>> g;
4     vector<int> dis, mL, mR;
5     explicit matching(int nL, int nr) : nL(nL), nr(nr), g(nL), dis(nL), mL(nL, -1),
6     mR(nr, -1) { }
7
8     void add(int l, int r) { // [i, j]
9         g[l].push_back(r);
10    }
11
12    void bfs() {
13        queue<int> q;
14        for(int u = 0; u < nL; u++) {
15            if(mL[u] == -1) q.push(u), dis[u] = 0;
16            else dis[u] = -1;
17        }
18        while(!q.empty()) {
19            int l = q.front(); q.pop();
20            for(int r : g[l]) {
21                if(mR[r] != -1 && dis[mR[r]] == -1)
22                    q.push(mR[r]), dis[mR[r]] = dis[l] + 1;
23            }
24        }
25    }
26
27    bool canMatch(int l) {
28        for(int r : g[l]) if(mR[r] == -1)
29            return mR[r] = l, mL[l] = r, true;
30        for(int r : g[l]) if(dis[l] + 1 == dis[mR[r]] && canMatch(mR[r]))
31            return mR[r] = l, mL[l] = r, true;
32        return false;
33    }
34
35    int maxMatch() {
36        int ans = 0, turn = 1;
37        while(turn) {
38            bfs(), turn = 0;
39            for(int l = 0; l < nL; l++) if(mL[l] == -1)
40                turn += canMatch(l);
41            ans += turn;
42        }
43        return ans;
44    }
45
46    pair<vector<int>, vector<int>> minCover() {
47        vector<int> L, R;
48        for (int u = 0; u < nL; ++u) {
49            if(dis[u] == -1) L.push_back(u);
50            else if(mL[u] != -1) R.push_back(mL[u]);
51        }
52        return {L, R};
53    }
54 };

```